

BERECHENBARKEIT UND KOMPLEXITÄT

Übungsblatt 10

Prof. Rossmanith
C. Grüne, T. Janßen, T. Koglin
Lehrstuhl für Informatik 1
RWTH Aachen

WS 24/25
07.01.2025
Abgabe: Di. 14.01., 16:30

Lösung

Tutoraufgabe 10.1

Wir betrachten das folgende Problem mit Formeln in disjunktiver Normalform (DNF).

DNF-SAT

Eingabe: Boole'sche Formel φ in DNF über den Variablen $x_1, \dots, x_n$.

Frage: Existiert eine Wahrheitsbelegung von $x_1, \dots, x_n$, die φ erfüllt?

Zeigen Sie, dass DNF-SAT unter der Annahme $P \neq NP$ nicht NP-vollständig ist.

Lösung:

Zeige, dass $\text{DNF-SAT} \in P$ gilt: Wäre DNF-SAT NP-vollständig, so würde $P = NP$ folgen, was der Annahme widerspricht.

Eine Formel φ in DNF hat die Form $\bigvee_i \bigwedge_j L_{ij}$ für Literale L_{ij} . Damit eine Wahrheitsbelegung φ erfüllt, genügt es, dass eine Konjunktion $\bigwedge_j L_{ij}$ erfüllt wird. Umgekehrt ist dies auch hinreichend: Wenn φ erfüllt ist, muss auch eine der Konjunktionen erfüllt sein. Für eine solche Konjunktion $\bigwedge_j L_{ij}$ von Literalen ist die Erfüllbarkeit einfach zu testen: Es müssen *alle* Literale L_{ij} wahr sein, d. h., die Literale geben die mögliche Belegung schon vor und es genügt, zu testen, ob diese widerspruchsfrei ist.

Damit erhält man direkt einen Linearzeitalgorithmus für DNF-SAT: Man iteriert über alle Konjunktionen (von Literalen) und testet, ob diese erfüllbar sind. Falls eine erfüllbare Konjunktion existiert, so akzeptiert man. Andernfalls verwirft man.

Tutoraufgabe 10.2

Es sei $G = (V, E)$ ein ungerichteter Graph. Eine Menge $D \subseteq V$ heißt *Dominating Set* von G , wenn für jeden Knoten in V gilt, dass er in D liegt oder zu einem Knoten in D benachbart ist.

Wir definieren nun das Problem DOMINATING SET.

DOMINATING SET (DS)

Eingabe: Ein ungerichteter Graph $G = (V, E)$ und eine natürliche Zahl k .

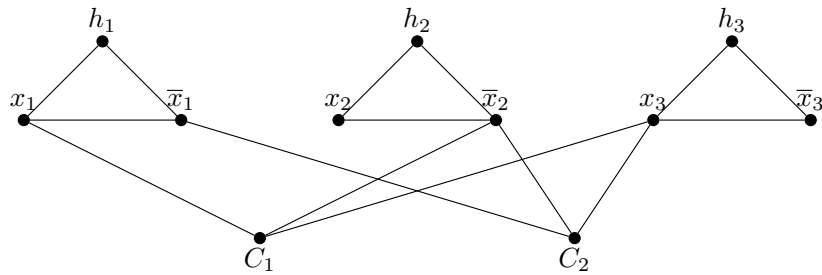
Frage: Hat G ein Dominating Set der Größe höchstens k ?

Zeigen Sie $\text{SATISFIABILITY} \leq_p \text{DOMINATING SET}$.

Lösung:

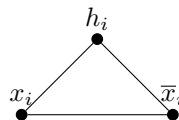
Wir zeigen dies durch eine Reduktion, die *Gadgets* benutzt. Gadgets simulieren dabei einen Teil des einen Problems, z.B. eine Variable in SAT, in dem anderen Problem, z.B. als Teilgraphen in DOMINATING SET. Am Ende werden alle Gadgets zusammengebaut, um die gesamte Instanz abzubilden. Dieses "Kochrezept" für Reduktionen ist vor allem hilfreich, um Reduktionen von SAT bzw. 3-SAT durchzuführen. Dafür werden Gadgets für die Variablen und für die Klauseln gebraucht als auch für die Verbindung von Variablen und Klauseln.

Wir beschreiben nun die konkrete Reduktion von SATISFIABILITY nach DOMINATING SET. Die resultierende DOMINATING SET-Instanz für die SAT-Instanz $(x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee x_3)$ sieht wie folgt aus. Wir setzen dabei $k = |X|$.



Wir schauen uns nun die einzelnen Gadgets an.

Wir fangen dafür mit den Gadgets für die Variablen an. Sei x_i eine Variable in SAT, dann bilden wir diese auf eine 3-Clique ab.

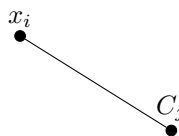


Die Funktion des Gadgets ist, dass genau der Knoten ausgewählt wird, der das jeweilige Literal darstellt. Da $k = |X|$ können wir für jede Variable genau einen Knoten auswählen. Wir müssen aber auch einen Knoten in der 3-Clique auswählen, um den Knoten h_i zu dominieren, da dieser nur zu x_i und $\bar{x}_i$ verbunden ist.

Jede Klausel C_j wird nun durch einen einzelnen Knoten dargestellt. Dieser muss nun von einem Knoten $x_i \in C_j$ dominiert werden, denn wir dürfen keinen weiteren Knoten mehr neben den $k = |X|$ Variablenknoten aufnehmen.



Als letztes müssen wir die Variablengadgets mit den Klauselgadgets verbinden. Wir konstruieren also ein Gadget für die Relation $x_i \in C_j$.



Dadurch kann nun auch der Knoten x_i die Klausel C_j dominieren.

($\Rightarrow$) Damit haben wir auch bereits die Idee der Korrektheit beschrieben. Wir beweisen diese nun explizit. Sei eine SAT-Instanz gegeben. Sei nun $X' \subseteq X$ die Lösung zu SAT, in der all auf "wahr" gesetzten Variablen enthalten sind. Wir kodieren diese Lösung $V'(X') \subseteq V$ in die DOMINATING SET-Instanz mit

$$V'(X') = \{x \mid x = x_i, \text{ falls } x_i \in X' \text{ und } \bar{x}_i, \text{ falls } x_i \notin X'\}.$$

Dies ist wiederum ein valides Dominating Set, denn alle Klauseln werden von den jeweiligen Knoten $x_i \in C_j$ dominiert, denn genau zwischen x_i und C_j existiert eine Kante. Zuletzt wird auch jedes h_i dominiert, da in jedem Variablengadget immer genau ein Knoten x_i bzw. $\bar{x}_i$ dominiert wird.

($\Leftarrow$) Betrachten wir nun die andere Richtung. Wir haben nun also eine Lösung V' für DOMINATING SET. Die Lösung für das Dominating Set ist maximal $k = |X|$. Daher muss nach dem Taubenschlagprinzip genau einer der drei Knoten von $x_i, \bar{x}_i, h_i$ in der Lösung sein (in einer 3-Clique muss einer der drei Knoten enthalten sein, damit alle Knoten, insbesondere h_i , der zu keinem anderen Knoten verbunden ist, auch dominiert werden). Es können nun keine weiteren Knoten, insbesondere die C_j , in der Lösung enthalten sein. Wir definieren also X' als Lösung für SAT wie folgt

$$X' = \left\{ x \mid x = \begin{cases} x_i, & \text{falls } x_i \vee h_i \in V' \\ \bar{x}_i, & \text{sonst} \end{cases} \right\}$$

Durch die Dominierung der Knoten C_j haben wir damit eine gültige SAT-Instanz.

Die Reduktion ist auch polynomiell berechenbar. Jedes Gadget ist konstant groß ist und für jede Variable, jede Klausel und jede Variable-Klausel-Relation. Dies ist polynomiell in der Eingabelänge.

Tutoraufgabe 10.3

Bei einem Scheduling Problem gibt es eine Anzahl von m Maschinen und n Jobs mit Laufzeiten $p_1, \dots, p_n$. Gesucht ist eine Zuweisung der n Jobs auf die m Maschinen, die dort dann hintereinander ausgeführt werden sollen. Ein Optimierungsziel ist es den „Makespan“ zu minimieren, dass heißt die maximale Zeit, die eine Maschine beschäftigt ist um die ihr zugewiesenen Jobs zu bearbeiten.

MAKESPAN SCHEDULING (MS)

Eingabe: m Maschinen, n Jobs mit Laufzeiten $p_1, \dots, p_n, b \in \mathbb{N}$

Frage: Gibt es eine Abbildung $s: \{1, \dots, n\} \rightarrow \{1, \dots, m\}$, sodass $\sum_{j: s(j)=i} p_j \leq b$ für alle $1 \leq i \leq m$?

Zeigen Sie: $\text{SUBSET-SUM} \leq_p \text{MAKESPAN-SCHEDULING}$.

Lösung:

Hinweis: Einen NP-Schwere-Beweis könnte man ganz einfach über BIN PACKING führen. Hier wurde jedoch die Reduktion von SUBSET-SUM verlangt.

Die Eingabe für SUBSET-SUM sei $((a_1, \dots, a_N), b)$. Wir konstruieren daraus eine Eingabe für MAKESPAN-SCHEDULING. Diese hat $N + 2$ Jobs. Für alle $1 \leq i \leq N$ setze die Laufzeiten der Jobs auf $p_i = a_i$. Weiterhin: $p_{N+1} = 2A - b$ und $p_{N+2} = A + b$, wobei $A = \sum_{i=1}^N a_i$. Die Anzahl der Maschinen setzen wir auf 2. Sei $2A$ der geforderte Makespan, also die maximale Laufzeit einer Maschine. Diese dadurch gegebene transformierende Funktion ist polynomial berechenbar.

Korrektheit:

($\Leftarrow$) Sei jetzt $((a_1, \dots, a_N), b) \in \text{SUBSET-SUM}$. Also gibt es eine Indexmenge $M \subseteq \{1, \dots, N\}$ mit $\sum_{i \in M} a_i = b$. Definiere die Zuordnung $s: \{1, \dots, N + 2\} \rightarrow \{1, 2\}$ mit

$$s(i) = \begin{cases} 1, & \text{falls } 1 \leq i \leq N \text{ und } i \in M \\ 2, & \text{falls } 1 \leq i \leq N \text{ und } i \notin M \\ 1, & \text{falls } i = N + 1 \\ 2, & \text{falls } i = N + 2 \end{cases}$$

Nachrechnen ergibt, dass dieser Schedule einen Makespan von $2A$ hat.

($\Rightarrow$) Sei die konstruierte MAKESPAN-SCHEDULING-Instanz eine Ja-Instanz, d.h., es gibt eine Zuordnung s von den n Jobs auf 2 Maschinen, so dass ein Makespan von höchstens $2A$ erreicht wird. Da $\sum_{1 \leq i \leq N+2} p_i = 4A$, haben beide Maschinen eine Last von exakt $2A$. Da $p_{N+1} + p_{N+2} = 3A \geq 2A$, sind Jobs $N + 1$ und $N + 2$ verschiedenen Maschinen zugeordnet. Die Jobs $M = \{1 \leq i \leq N \mid s(i) = s(N + 1)\}$, die der Maschine mit Job $N + 1$ zugeordnet sind, haben eine Last von $2A - (2A - b) = b$. Also ist M eine Lösung von der ursprünglichen SUBSET-SUM-Instanz.